SOFTWARE ENGINEERING LAB TASK 3

11-12-2024

HU22CSEN0100287

SAI GANESH ESWARAPRASAD

HU22CSEN0100287

Introduction to JIRA Software

JIRA is a proprietary software developed by Atlassian, widely used for:

- **Bug tracking**, **issue tracking**, and **agile project management**.
- **Planning, tracking, and releasing** software projects.
- Providing a centralized platform to manage tasks and organize work.
- Integrating with other development tools and offering customizable features.

Uses of JIRA Software

1. **Agile Project Management**
    - Supports Scrum (sprints) and Kanban (continuous flow).
    - Helps create user stories, manage sprints, and track progress.

2. **Bug and Issue Tracking**
    - Logs, prioritizes, and resolves bugs efficiently.
    - Assigns tasks with deadlines to team members.
3. **Task Management**
    - Breaks projects into manageable tasks.
    - Monitors status through workflows.
4. **Software Development**
    - Tracks code changes and manages version control.
    - Integrates with CI/CD tools for streamlined processes.
5. **Tracking and Reporting**
    - Dashboards and charts monitor project performance.
6. **Team Collaboration**
    - Centralizes project data to ensure transparency and alignment.

Key Features of JIRA

- **Issue tracking** and **custom workflows**.
- **Reporting and analytics** tools.
- Integrations with other tools.
- **Automation** capabilities.
- Task prioritization and user management.

HU22CSEN0100287

JIRA Scheme

A JIRA scheme is a collection of values applied to one or more projects. Examples include:

1. **Issue Type Scheme**: Defines different types of work items.
2. **Notification Scheme**: Configures notifications for changes.
3. **Permission Scheme**: Manages user permissions for accessing resources.
4. **Issue Security Scheme**: Controls issue visibility.

System Administration in JIRA

JIRA provides several features for system administration:

- **Audit Log**: Tracks issue history and modifications.
- **Issue Linking**: Links related issues for better tracking.
- **Mail Integration**: Allows sending issues through POP or IMAP servers.
- **Events**: Manages templates, notifications, and workflow transitions.
- **Watch List**: Enables users to monitor updates on specific issues.

JIRA Issues and Issue Types

- **Issue**: A unit of work to be tracked and managed.
- **Issue Types**: Categorize and manage work effectively.

HU22CSEN0100287

- Bug: Software error or flaw.
- Improvement: Enhancing existing features.
- Task: Specific work to be completed.
- Epic: Large, high-level work divided into smaller components.
- Story: User-centric requirements or features.
- Incident: Unexpected disruptions or failures.
- Sub-task: Smaller work items under a parent issue.

JIRA Issue Attributes

- **Summary**: Brief title of the issue.
- **Description**: Detailed explanation.
- **Status**: Current progress (e.g., Open, In Progress).
- **Assignee**: Responsible team member.
- **Reporter**: Person who reported the issue.
- **Priority**: Importance (High, Medium, Low).
- **Labels**: Tags for organization and filtering.
- **Components**: Specific project sections.
- **Fix Version**: Target version for resolution.

HU22CSEN0100287

JIRA Software Statuses

Statuses reflect issue progress:

- Open: Newly created, unassigned issue.
- In Progress: Work started.
- Resolved: Fixed but not verified.
- Closed: Finalized issue.
- Reopened: Issue reopened after resolution.
- Pending: Waiting for input.
- On Hold: Temporarily paused.
- Deferred: Postponed for later.
- Done: Ready for closure.

JIRA Components

- Represent core features of JIRA.
- Details include name, description, lead, and associated issues.
- Manage components via the project's Components page.

Sub-Tasks in JIRA

- Break parent issues into smaller tasks for better organization.
- Created as sub-tasks or by converting existing issues.

HU22CSEN0100287

Workflows in JIRA

Workflows define the lifecycle of an issue, including:

- **Statuses**: Positions of issues (e.g., Open, Resolved).
- **Transitions**: Movement between statuses.
- **Conditions**: Control who can transition issues.
- **Validators**: Ensure transitions are valid.
- **Assignee**: Responsible person.

Plug-ins in JIRA

Plug-ins enhance functionality with additional features. Examples include:

- Zendesk, Salesforce, GitHub.
- Enable direct issue reporting and management.

JIRA Agile

Supports Agile methodologies with components like:

- Agile Board: Tracks sprint progress.
- Backlog: Plans and estimates stories.
- Workflow: Maps columns to workflow statuses.

HU22CSEN0100287

Clone and Link in JIRA

- **Cloning Issues**: Enables teams to work independently on the same issue.
- **Linking Issues**: Tracks related issues across servers.

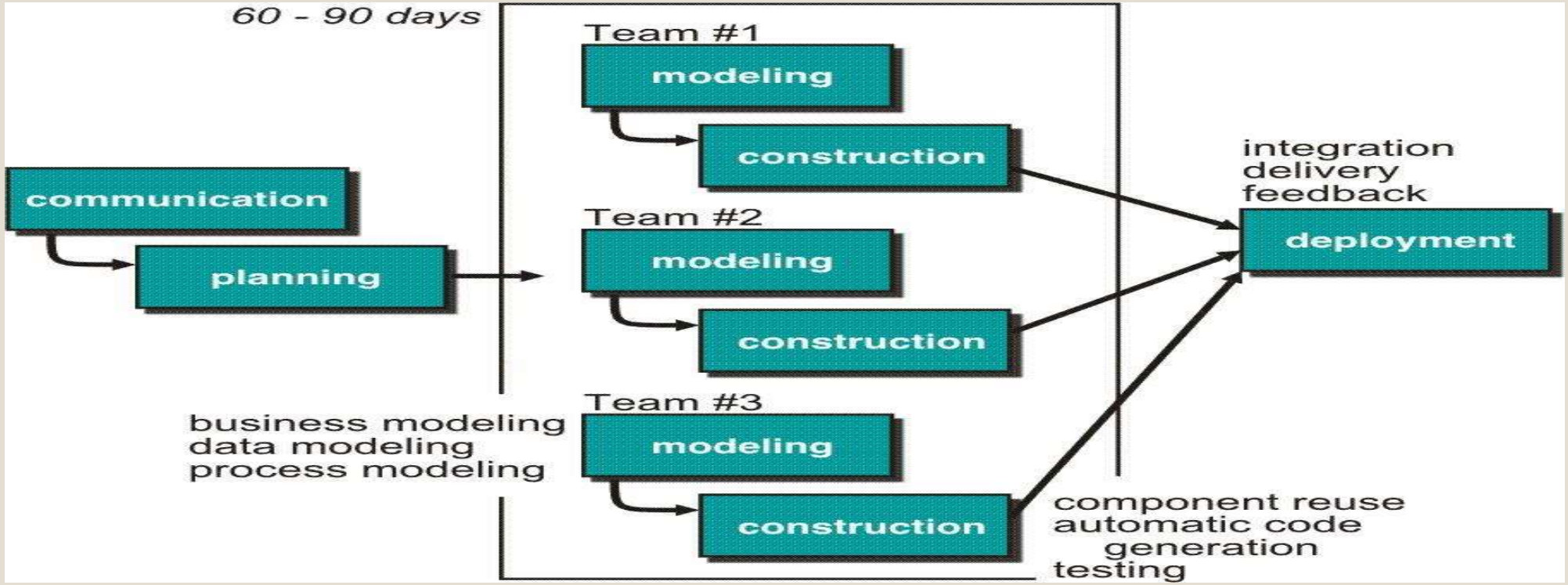
Reports in JIRA

JIRA offers multiple reporting options for tracking progress:

- **Burndown Chart**: Displays work completed vs. remaining.
- **Velocity Chart, Control Chart, Sprint Report, Epic Report**: Provide project insights.

HU22CSEN0100287

Rapid Application Development (RAD) Model



Makes heavy use of reusable software components with an extremely short development cycle

RAD model

- **Communication** – to understand problem information
- **Planning** – multiple s/w teams works in parallel on diff. system.
- **Modeling –**
  - **Business modeling –** Information flow among business is working.

  Ex. What kind of information drives?

  Who is going to generate information?

  From where information comes and goes?
  - **Data modeling**
  - **Process modeling**

RAD

- **Construction –** it highlighting the use of pre-existing software component
- Automatic code generation is done..
- **Deployment** – Deliver to customer basis for subsequent iteration.
- RAD model emphasize a short development cycle.
- "High speed" edition of linear sequential model.
- If requirement are well understood and project scope is constrained then it enable development team to create " fully functional system" within a very short time period

RAD Model **Drawback**

- Large projects more people are required & large teams
- If developer & customer are not committed RAD fails
- Performance is major issue then RAD fails
- RAD is suitable where risk is low
- RAD requires sufficient human resources
- Not appropriate when technical risks are there

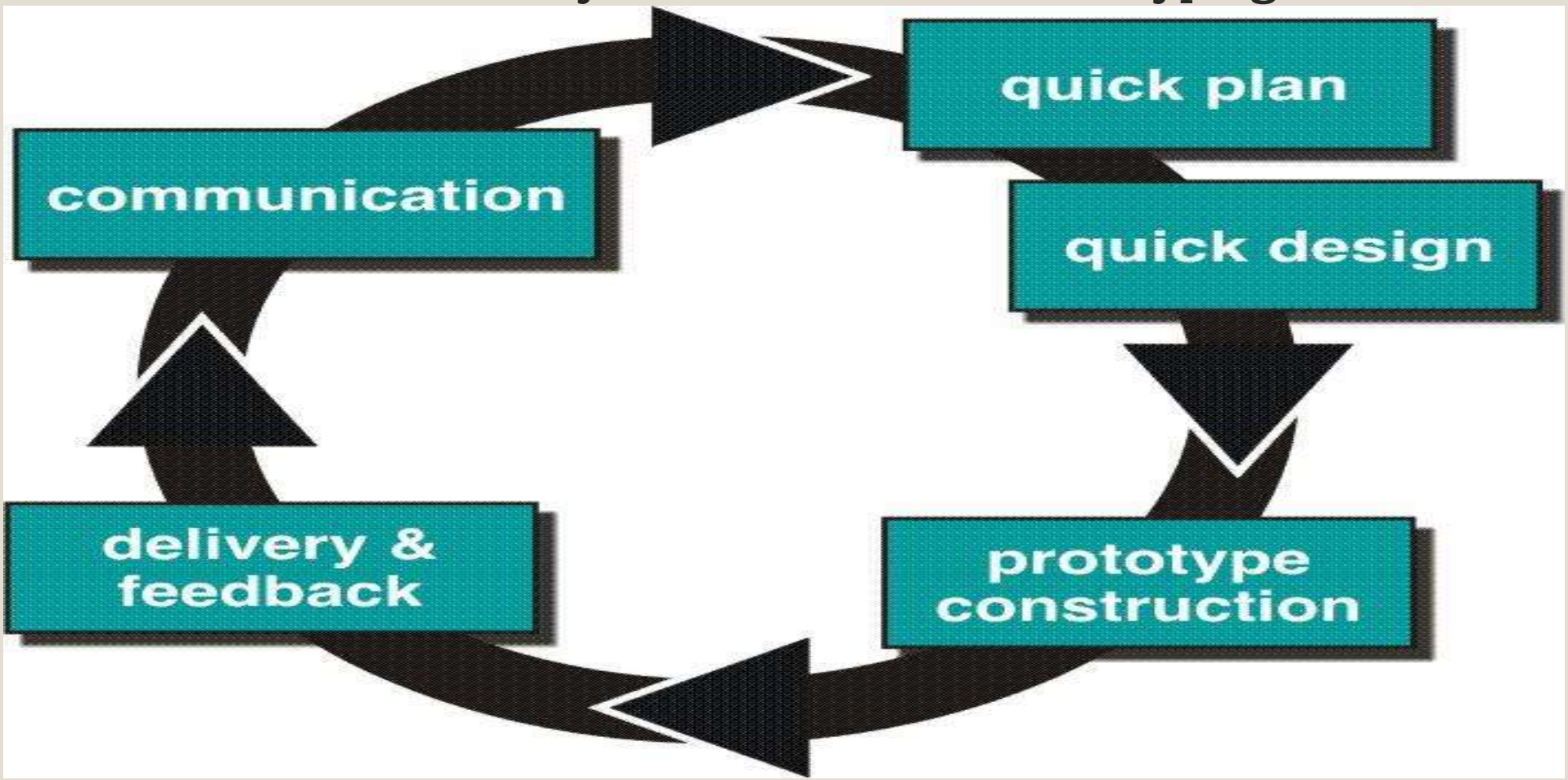
Evolutionary Process Model: Prototyping Model

- Starts with communication
- Requirement gathering is done
- Software developer use this model when requirements are fuzzy
- Quick design is done leads to prototype
- Prototype is refined until customer is satisfied
- Programs are built and thrown on it and we rebuild the system to improve quality

Drawbacks:

- When developer is unsure of efficiency algorithm
- When developer has some compromises like in appropriate programming languages, os used, algorithms used

Evolutionary Process Models : Prototyping



Prototyping Best approach when:

- customer Objectives are general but does not have details like input, processing, or output requirement.
- Developer may be unsure of the efficiency of an algorithm, O.S., or the form that human machine interaction should take.
- when requirement are fuzzy.
- Quick design leads to prototype construction.
- Prototype is deployed and evaluated by the customer/user.
- Feedback from customer/end user will refine requirement and that is how iteration occurs during prototype to satisfy the needs of the customer.

Prototyping

- Prototype can be serve as “the first system”.
- Both customers and developers like the prototyping paradigm.
  - Customer/End user gets a feel for the actual system
  - Developer get to build something immediately.

Limitations Of SDLC Process Models

- ❖Difficulty To Adapt The Changes
- ❖ Less Flexibility
- ❖Long Software Development Cycles.
- ❖To overcome these challenges, the Agile method was introduced.
- ❖ The Scrum model is also one of the Agile frameworks

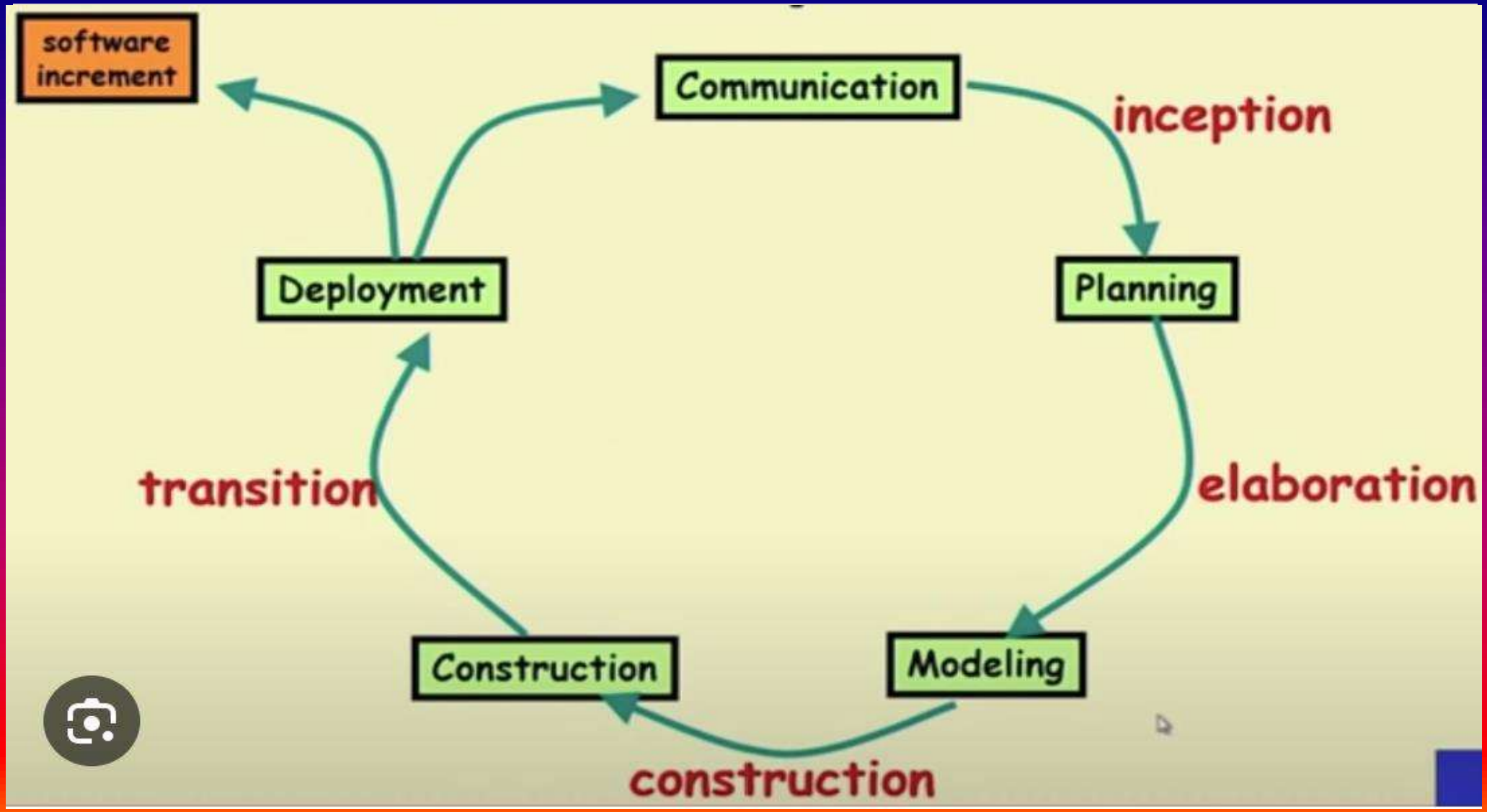
Unified Process Model

It is rational unified process model developed by grady booch

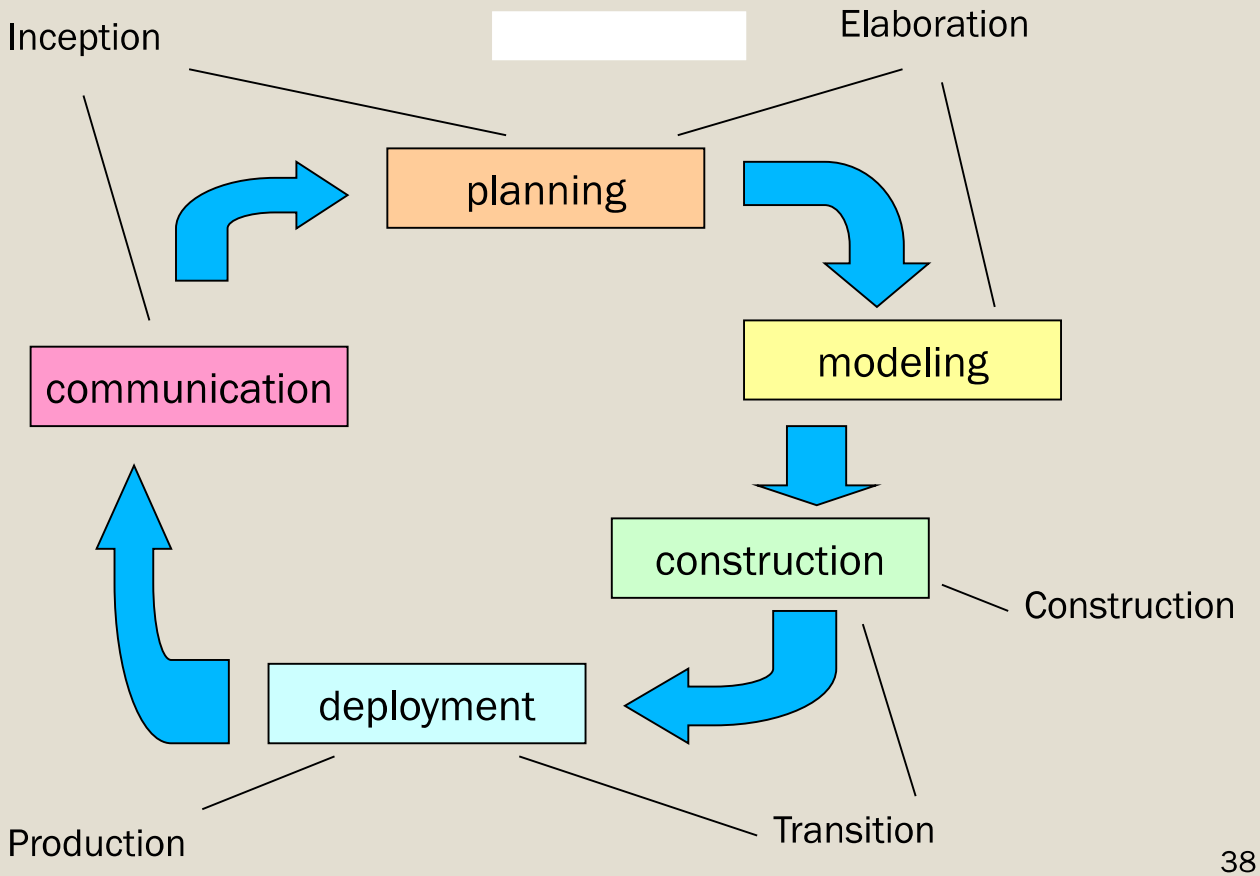
It follows iterative incremental approach

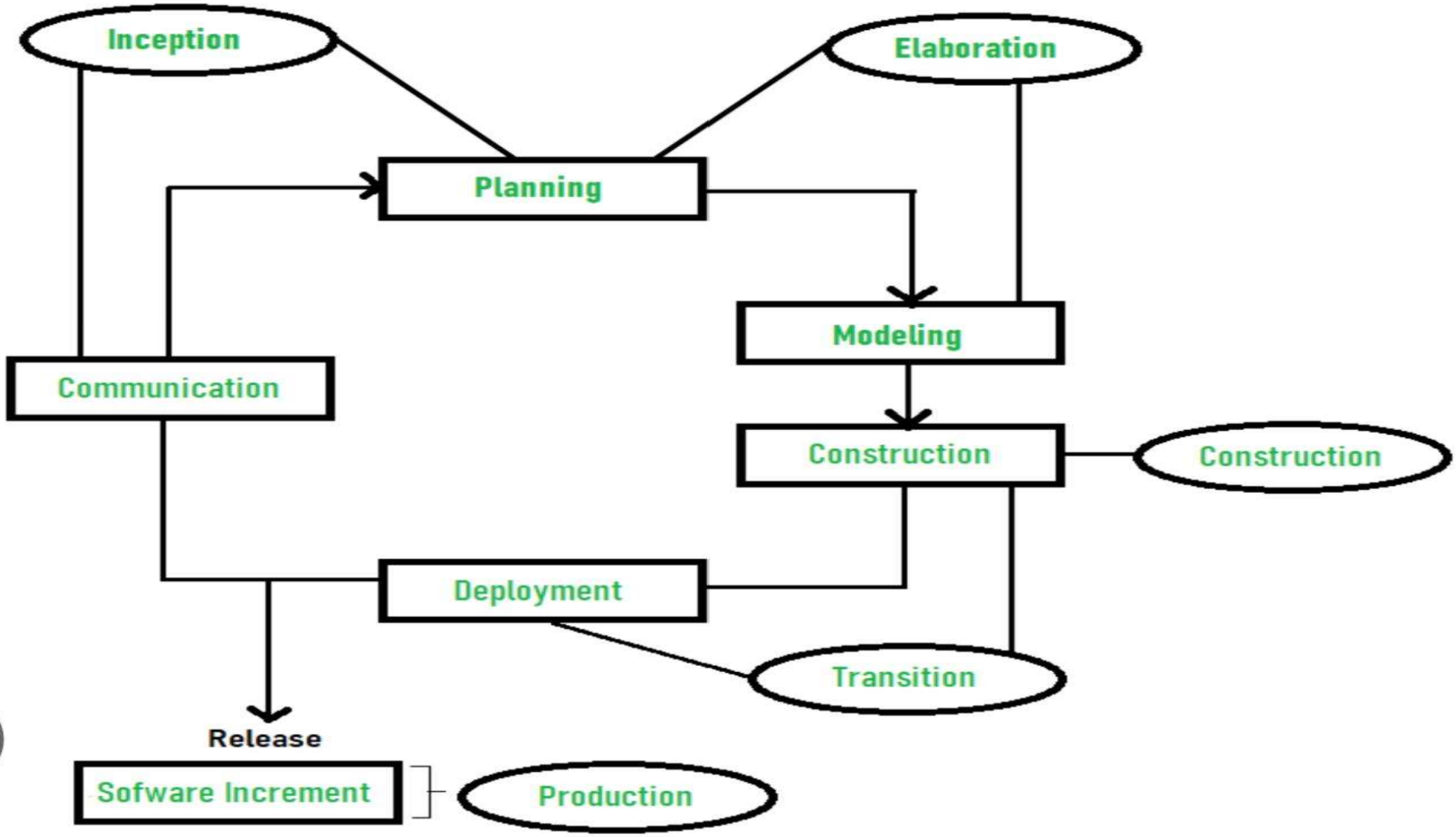
It has 5 phases

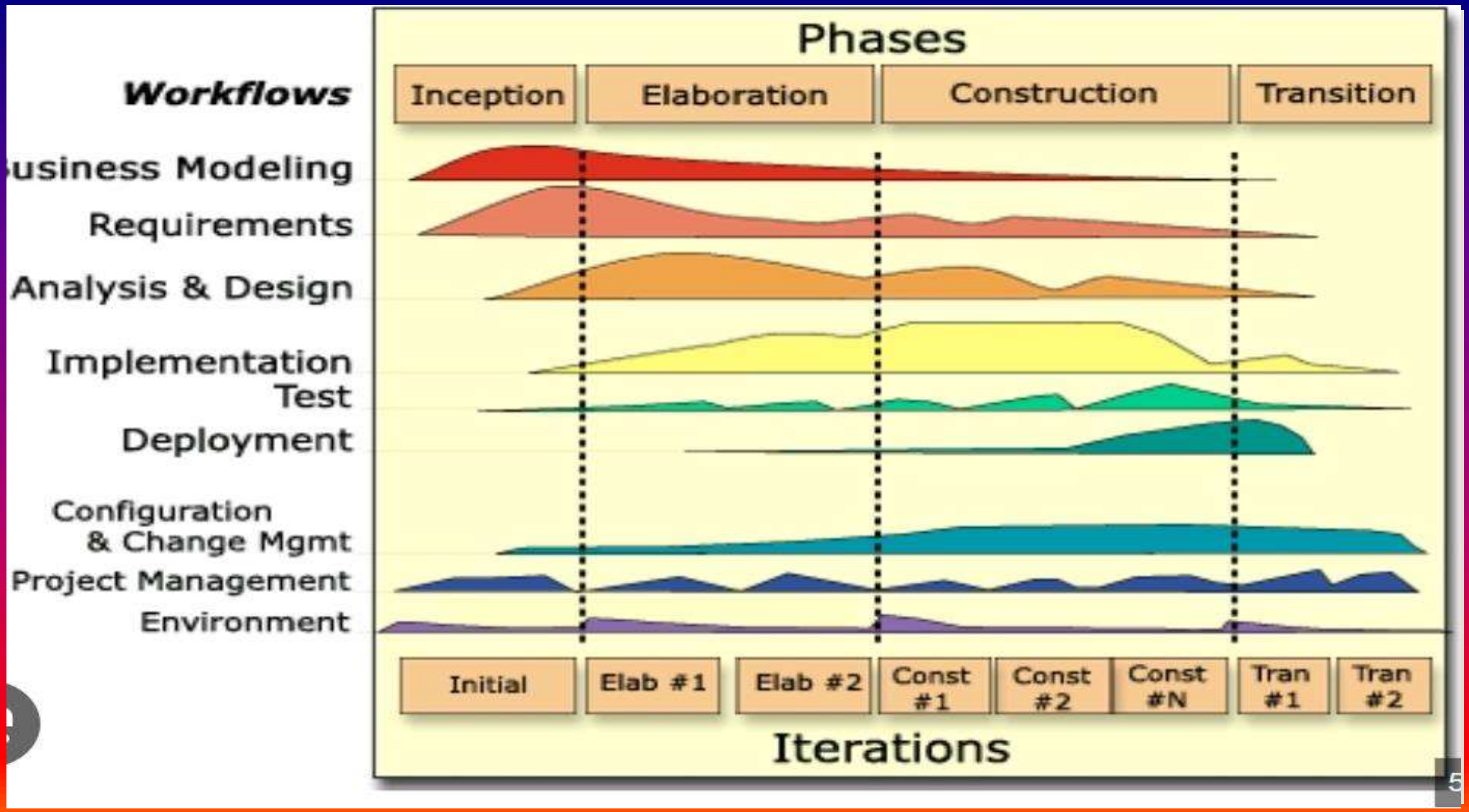
- Inception
- Elaboration
- Construction
- Transition
- production


software increment
Communication
inception
Planning
elaboration
Modeling
construction
Construction
transition
Deployment

Phases of the Unified Process




Inception
Elaboration
Planning
Modeling
Communication
Construction
Construction
Deployment
Transition
Release
Sofware Increment
Production


Phases
Workflows
Inception
Elaboration
Construction
Transition
usiness Modeling
Requirements
Analysis & Design
Implementation
Test
Deployment
Configuration
& Change Mgmt
Project Management
Environment
Initial
Elab #1
Elab #2
Const #1
Const #2
Const #N
Tran #1
Tran #2
Iterations

Inception phase

Vision document
Initial use-case model
Initial project glossary
Initial business case
Initial risk assessment.
Project plan,
phases and iterations.
Business model,
if necessary.
One or more prototypes

Elaboration phase

Use-case model
Supplementary requirements
including non-functional
Analysis model
Software architecture
Description.
Executable architectural
prototype.
Preliminary design model
Revised risk list
Project plan including
iteration plan
adapted workflows
milestones
technical work products
Preliminary user manual

Construction phase

Design model
Software components
Integrated software
increment
Test plan and procedure
Test cases
Support documentation
user manuals
installation manuals
description of current
increment

Transition phase

Delivered software increment
Beta test reports
General user feedback